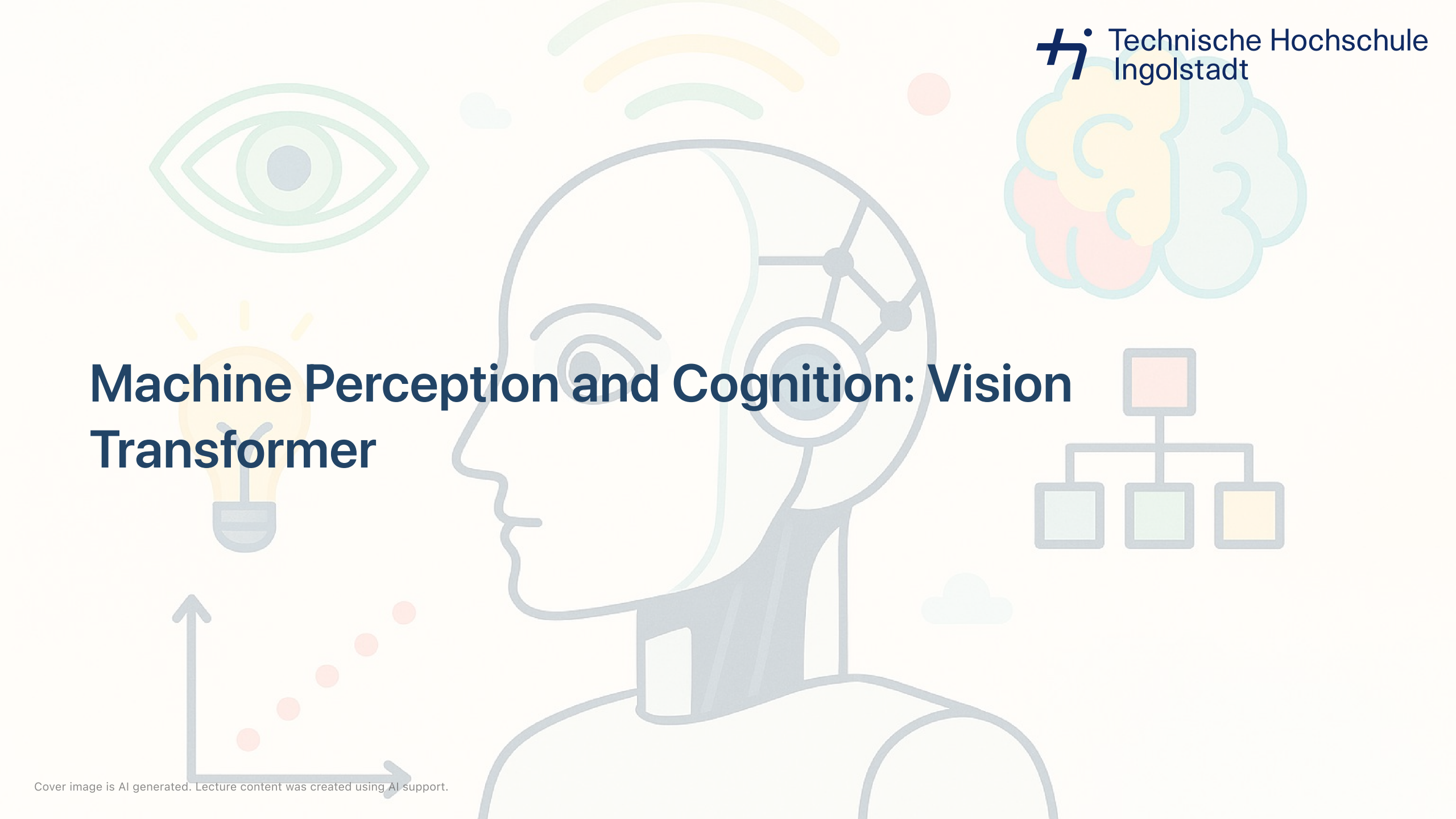
Technische Hochschule Ingolstadt

# Machine Perception and Cognition: Vision Transformer

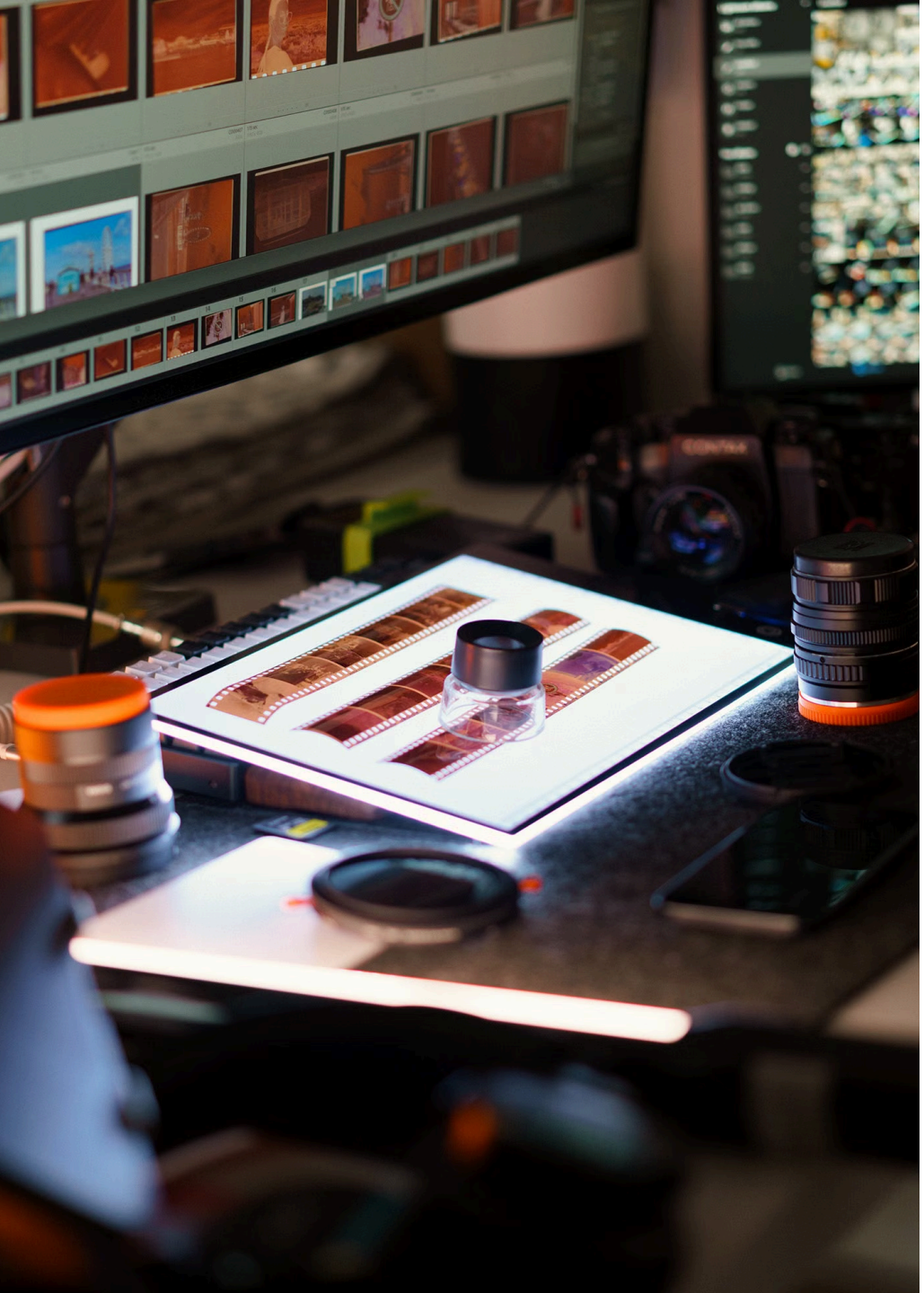


Cover image is AI generated. Lecture content was created using AI support.

# Learning Goals

After this lecture you will be able to:

- Explain how images are converted into **patch sequences** for transformer processing
- Describe the full **Vision Transformer (ViT) architecture** — patch embedding, class token, positional encoding, encoder blocks, and classification head
- Explain how **self-attention operates on image patches** and why attention maps are interpretable
- Compare the inductive biases of **CNNs and ViT**
- Describe **five self-supervised and supervised pre-training strategies** for ViT: supervised classification, masked autoencoder, SimCLR, CLIP, and DINOv2



# Vision Transformer

So how do we go from a transformer processing text to a transformer processing images?

Image source: https://www.pexels.com/photo/professional-photography-workspace-with-film-development-tools-37204672/

# From Tokens to Patches

The Transformer we studied in the previous chapter expects a **sequence of vectors** as input:

$$\mathbf{Z} = (\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_n) \in \mathbb{R}^{n \times d}$$

For text, each $\mathbf{z}_i$ is a token embedding. The core question for visual data is:

> **How do we turn an image into a sequence of vectors?**

# From Tokens to Patches

**Option 1 — pixel sequence:** flatten the image $H \times W \times C$ into one long sequence. For a $224 \times 224$ RGB image this gives $150\,528$ tokens — prohibitively long, because self-attention computes a score between every pair of tokens, so memory and compute grow with the square of the sequence length.

**Option 2 — patch sequence:** split the image into non-overlapping patches and treat each patch as one token. A $16 \times 16$ patch on a $224 \times 224$ image gives only $196$ tokens. This is the key idea of the **Vision Transformer (ViT)**.

# Image Patches as Tokens

An image $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$ is divided into $N$ non-overlapping patches, each of size $P \times P$:

$$N = \frac{H \cdot W}{P^2}$$

Each patch $\mathbf{p}_i \in \mathbb{R}^{P^2 C}$ is flattened into a vector and linearly projected to the model dimension $d$:

$$\mathbf{z}_i^{(0)} = \mathbf{p}_i \mathbf{E}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 C) \times d}$$

The projection matrix $\mathbf{E}$ is the **patch embedding layer** — a learned linear map trained jointly with the rest of the model.

For $H = W = 224$, $P = 16$, $C = 3$: $N = 196$ patches, each of dimension $P^2 C = 768$ before projection.

# Vision Transformer

Principle of a vision transformer, processing an image as a sequence of patches:

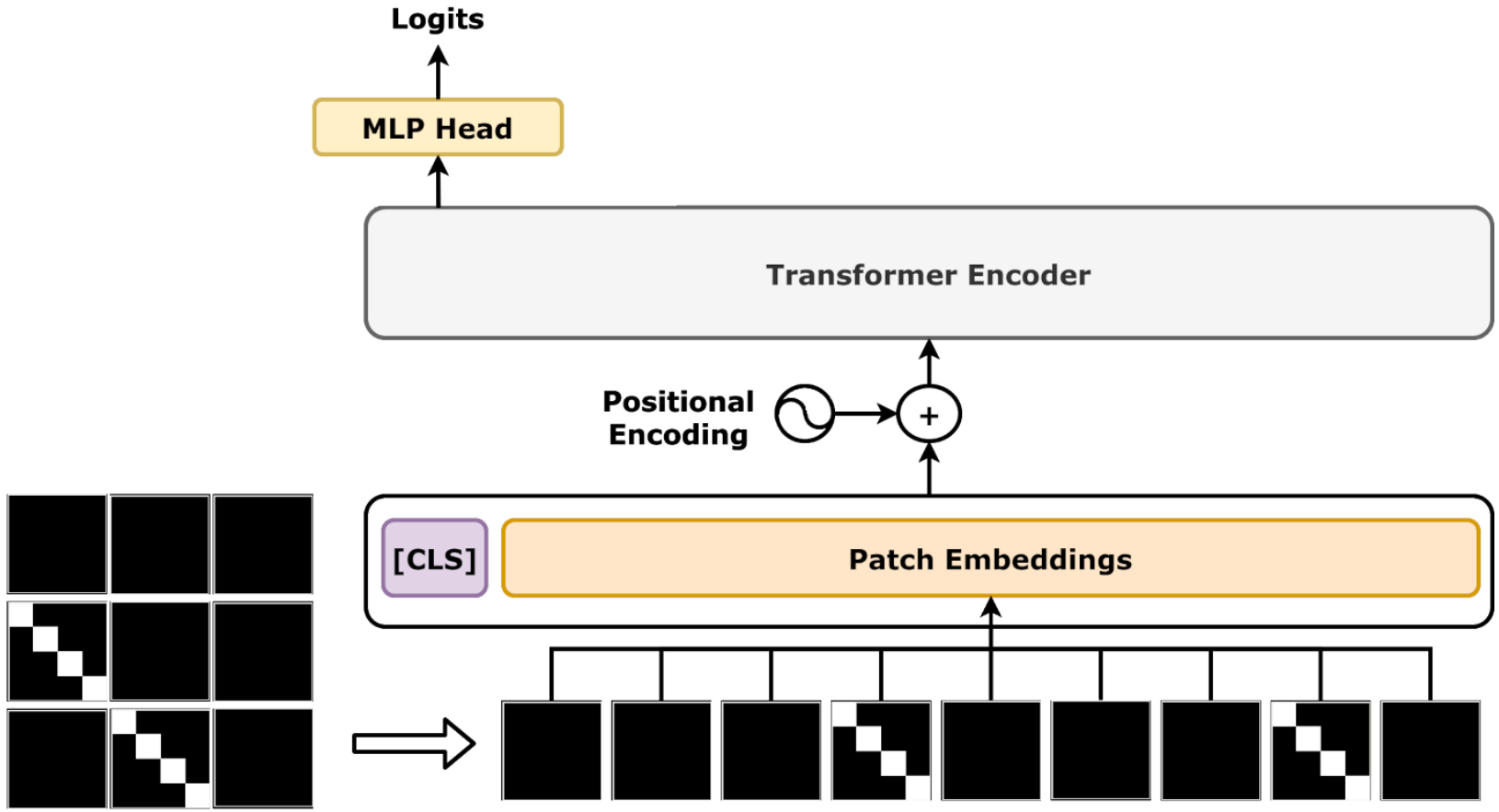



Image source: Daniel Voigt Godoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Vision_Transformer.png

# Vision Transformer

Vision transformer with a dog+cat image as input:

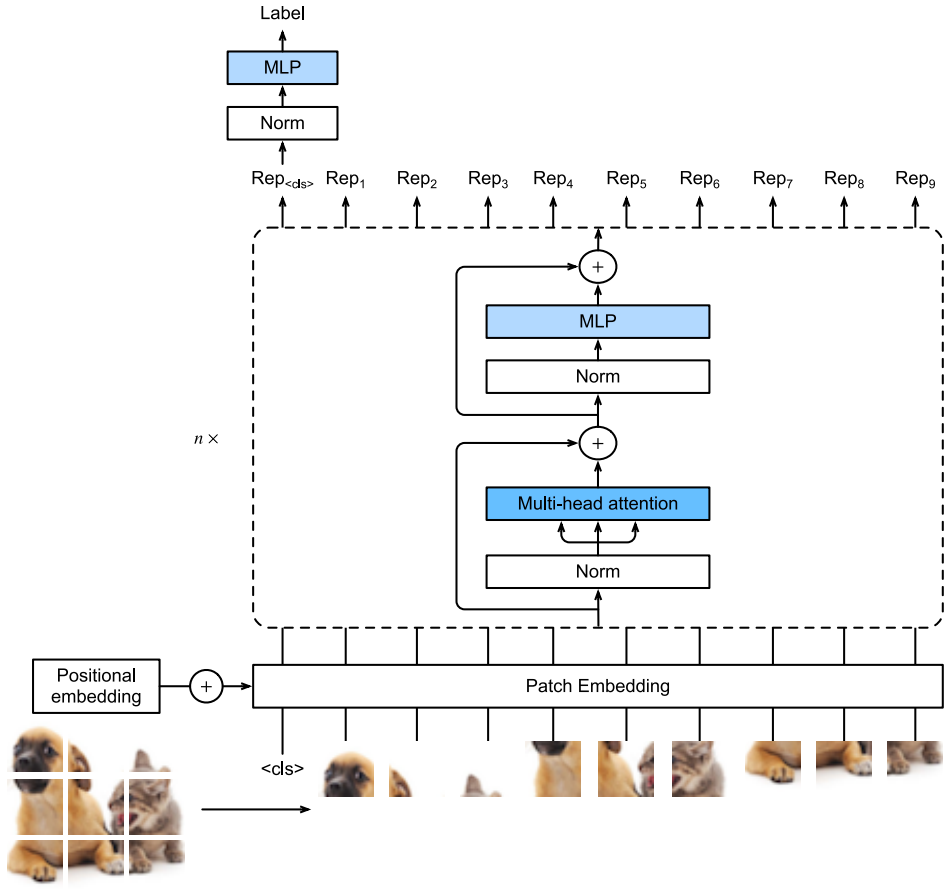



Image source: Zhang, Aston and Lipton, Zachary C. and Li, Mu and Smola, Alexander J., CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Vision_Transformer.svg

# The Class Token

ViT prepends a special learnable **class token** $\mathbf{z}_{\text{cls}} \in \mathbb{R}^d$ to the patch sequence:

$$\mathbf{Z}^{(0)} = \left[\mathbf{z}_{\text{cls}},\ \mathbf{z}_1^{(0)},\ \mathbf{z}_2^{(0)},\ \ldots,\ \mathbf{z}_N^{(0)}\right] \in \mathbb{R}^{(N+1)\times d}$$

**Purpose of the class token:**

- It is not tied to any specific patch — it acts as a global summary token
- After $L$ transformer encoder layers, the output at position $0$ (i.e., $\mathbf{z}_{\text{cls}}^{(L)}$) aggregates global image information through attention
- A classification head (MLP) is applied to $\mathbf{z}_{\text{cls}}^{(L)}$ to predict the class label

The class token learns to attend to the most discriminative patches across all layers.

# Positional Encoding for Patches

Self-attention is permutation-equivariant — without positional information, the model cannot distinguish between different spatial arrangements of patches.

ViT adds **learnable 1D positional embeddings** $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times d}$ to the patch embeddings:

$$\mathbf{Z}^{(0)} \leftarrow \mathbf{Z}^{(0)} + \mathbf{E}_{\text{pos}}$$

Each of the $N + 1$ positions (class token + $N$ patches) gets its own learned position vector.

# Vision Transformer

Vision transformer as in the original paper:

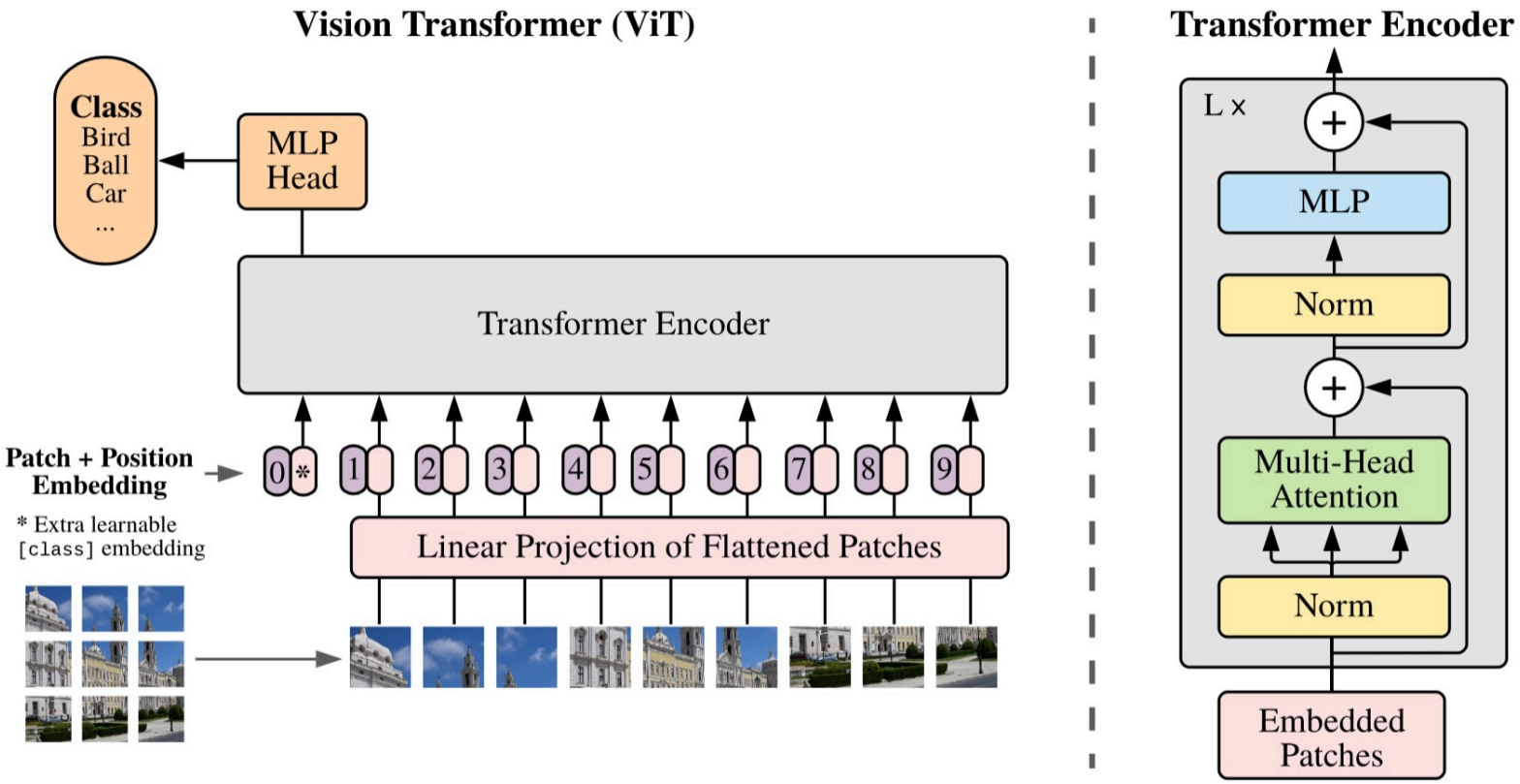



Image source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.

# ViT Classification Head

After $L$ encoder layers, the class token representation $\mathbf{z}_{\text{cls}}^{(L)} \in \mathbb{R}^d$ is passed to an **MLP head**:

$$\hat{\mathbf{y}} = \text{softmax}\Big(\mathbf{W}_2\, \sigma(\mathbf{W}_1\, \mathbf{z}_{\text{cls}}^{(L)} + \mathbf{b}_1) + \mathbf{b}_2\Big)$$

During pre-training, ViT uses a classification head (linear or MLP, as shown here). During fine-tuning, this head is replaced with a task-specific head, typically a new linear layer.

**Alternative to class token — global average pooling:**
Instead of using $\mathbf{z}_{\text{cls}}^{(L)}$, some ViT variants average the $N$ patch tokens after the last encoder layer:

$$\mathbf{z}_{\text{gap}}^{(L)} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{z}_i^{(L)}$$

# Class Token Aggregation

The class token plays a central role in information aggregation:

**Layer 0** (input): $\mathbf{z}_{\text{cls}}^{(0)}$ is a randomly initialized learnable vector — it contains no image information yet.

**After each layer** $\ell$: the class token aggregates information from all patch tokens through cross-token attention. Its representation is progressively refined.

**After layer** $L$: $\mathbf{z}_{\text{cls}}^{(L)}$ has attended to all patches across $L$ rounds of multi-head self-attention — it encodes a rich, global summary of the image that is used for classification.

# ViT Model Variants

The original ViT paper (Dosovitskiy et al., 2020) defines three standard model sizes:

| Variant | Layers $L$ | Hidden dim $d$ | Heads | Params |
|---|---|---|---|---|
| ViT-B (Base) | 12 | 768 | 12 | 86M |
| ViT-L (Large) | 24 | 1024 | 16 | 307M |
| ViT-H (Huge) | 32 | 1280 | 16 | 632M |

# CNN vs. ViT: Inductive Biases

**CNNs** build in strong spatial inductive biases:

- **Translation equivariance**: applying a filter at different positions yields the same response
- **Locality**: filters only see a local receptive field at each step
- These biases are effective when training data is limited

# CNN vs. ViT: Inductive Biases

**ViT** has almost no spatial inductive bias:

- Self-attention has a **global receptive field** from the first layer — every patch can attend to every other patch
- Only 2D structure used: patch extraction and positional encoding
- Without inductive bias, ViT needs much more data to learn spatial relationships

**Consequence:** ViT underperforms CNNs on small datasets but surpasses them when pre-trained at scale — the model learns spatial structure from data.

# Self-Attention on Image Patches

In each ViT encoder layer, multi-head self-attention is applied to the sequence of $N+1$ tokens (patches + class token):

For each attention head $h$ with projection matrices $\mathbf{W}_h^Q, \mathbf{W}_h^K, \mathbf{W}_h^V \in \mathbb{R}^{d \times d_k}$:

$$\mathbf{Q}_h = \mathbf{Z}\mathbf{W}_h^Q, \quad \mathbf{K}_h = \mathbf{Z}\mathbf{W}_h^K, \quad \mathbf{V}_h = \mathbf{Z}\mathbf{W}_h^V$$

$$\mathbf{A}_h = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{(N+1)\times(N+1)}$$

The $(i, j)$-entry of $\mathbf{A}_h$ is the **attention weight** from patch $i$ to patch $j$ — how much patch $i$ attends to the content of patch $j$.

No causal masking is applied — all patches can attend to all others.

# Multi-Head Attention: Different Perspectives

With $h$ attention heads, each head learns a **different attention pattern**:

- Some heads may focus on **local texture** within neighboring patches
- Other heads capture **long-range semantic relationships** (e.g., relating a head to a body)
- Some heads specialize in **background suppression**

This multi-head mechanism allows the model to jointly attend to information from different representation subspaces at different positions — directly analogous to how different CNN filters detect different features.

# Attention Maps in ViT

The attention weight matrix $\mathbf{A}_h^{(\ell)} \in \mathbb{R}^{(N+1)\times(N+1)}$ at layer $\ell$ can be visualized as a **spatial attention map** over the patch grid.

For the class token (row 0), the weights across the $N$ patch positions reveal which patches the class token directly attends to at that layer. Averaging over all $n_h$ heads gives a single per-layer map:

$$\mathbf{A}^{(\ell)} = \frac{1}{n_h} \sum_{h=1}^{n_h} \mathbf{A}_h^{(\ell)}, \qquad a_{0,j}^{(\ell)} = [\mathbf{A}^{(\ell)}]_{0,j}, \quad j = 1, \ldots, N$$

# Attention Maps in ViT

**Limitation**: a single-layer map captures only *direct* attention at layer $\ell$. It ignores how information propagated through earlier layers — the class token may attend to a patch that itself attended to relevant regions in layer $\ell - 1$.

# Residual Connections as Part of Attention

When visualizing attention, do not forget the residual connections!

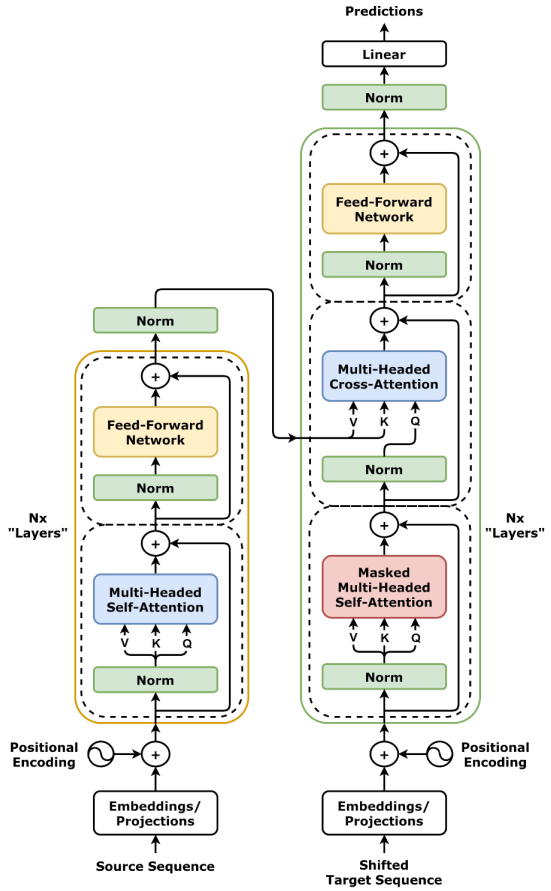



Image source: dvgodoy, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Transformer,_full_architecture.png

# Attention Rollout

**Attention Rollout** (Abnar & Zuidema, 2020) addresses this by propagating attention *recursively* from the last layer back to the input, accounting for the residual connection in each block.

**Residual connection as identity attention:** Each transformer block applies attention and then adds the original input via a skip connection. For token representations this means:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell)}\,\mathbf{Z}^{(\ell-1)} + \mathbf{I}\,\mathbf{Z}^{(\ell-1)} = \left(\mathbf{A}^{(\ell)} + \mathbf{I}\right)\mathbf{Z}^{(\ell-1)}$$

# Attention Rollout

The identity $\mathbf{I}$ models the residual path — every token always "attends to itself" through the skip connection. Because both $\mathbf{A}^{(\ell)}$ (rows sum to 1) and $\mathbf{I}$ (rows sum to 1) contribute, the combined matrix has rows summing to 2. Row-normalizing restores a proper attention distribution:

$$\tilde{\mathbf{A}}^{(\ell)} = \frac{\mathbf{A}^{(\ell)} + \mathbf{I}}{2} = 0.5\,\mathbf{A}^{(\ell)} + 0.5\,\mathbf{I}$$

Starting from $\hat{\mathbf{A}}^{(0)} = \mathbf{I}$, the rolled-out attention is built recursively for $\ell = 1, \ldots, L$:

$$\hat{\mathbf{A}}^{(\ell)} = \tilde{\mathbf{A}}^{(\ell)} \cdot \hat{\mathbf{A}}^{(\ell-1)}$$

Row 0 of the final matrix $\hat{\mathbf{A}}^{(L)}$ gives the **effective contribution** of each input patch to the class token after $L$ layers of transitive attention flow.

# Attention Rollout

**Upscaling**: extract $\hat{\mathbf{A}}^{(L)}_{0,\,1:N}$ ($N$ scalar weights), reshape to $\frac{H}{P} \times \frac{W}{P}$, and bilinearly upsample to $H \times W$ to obtain a full-resolution interpretability map.

Abnar, S. & Zuidema, W. (2020). Quantifying Attention Flow in Transformers. Proceedings of ACL 2020. arXiv:2005.00928

# Attention in a Vision Transformer

Some examples with input image left and attention map right:

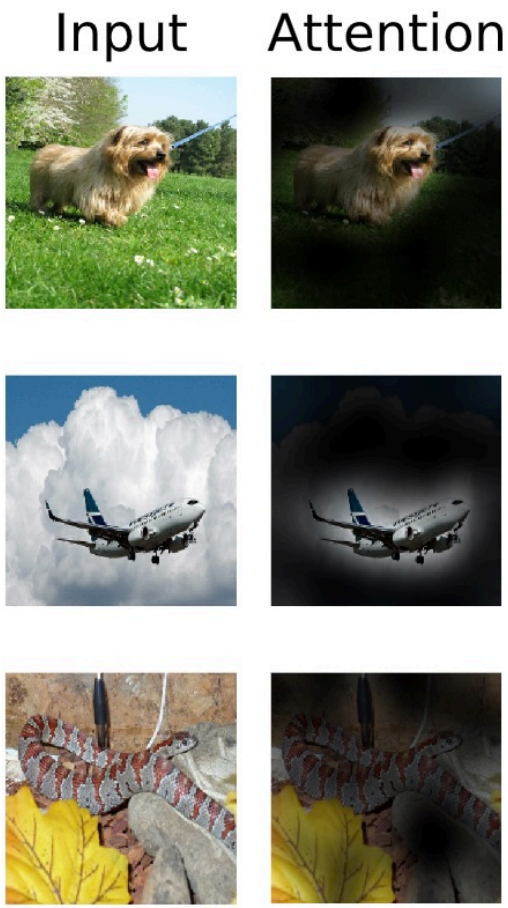



Image source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.

# Attention in a Vision Transformer

Some more examples:

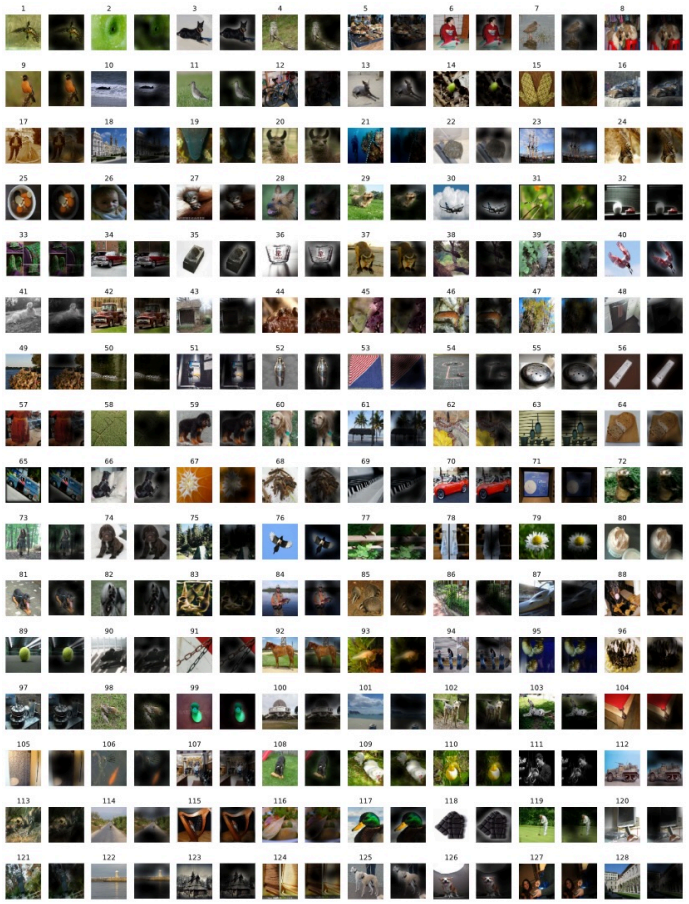


Image source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929.



# Pre-Training a Vision Transformer

How can we pre-train a vision transformer?

Image source: https://www.pexels.com/de-de/foto/fragezeichen-illustration-356079/

# Why Pre-Training?

Training a large ViT from scratch on a small task-specific dataset overfits quickly due to the **lack of inductive bias**.

**Pre-training** first trains the model on a large dataset or with a self-supervised objective, then **fine-tunes** the resulting model on a smaller downstream task:

$$\underbrace{\text{Large data / self-supervised}}_{\text{pre-training}} \xrightarrow{\text{fine-tune}} \underbrace{\text{small labeled task}}_{\text{downstream}}$$

# Supervised Pre-Training on ImageNet

The most straightforward pre-training: train ViT with a **cross-entropy classification loss** on ImageNet:

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

where $K$ is the number of classes (1000 for ImageNet-1k, 21,841 for ImageNet-21k).

Class probabilities $\hat{y}_k$ are estimated by placing a classification head (typically an MLP during supervised pre-training) on the final class-token representation.

# Supervised Pre-Training: Transfer

After ImageNet pre-training, the ViT classification head is replaced and the model is fine-tuned on a downstream task:

1. Remove the pre-training classification head
2. Attach a task-specific head (e.g., a linear layer for a new classification task)
3. Fine-tune the full model (or just the head) with a small learning rate

# Masked Autoencoder

The MAE encoder and decoder are both stacks of transformer encoder blocks. The decoder is used only for reconstruction during pre-training, making the concept similar to a denoising autoencoder:

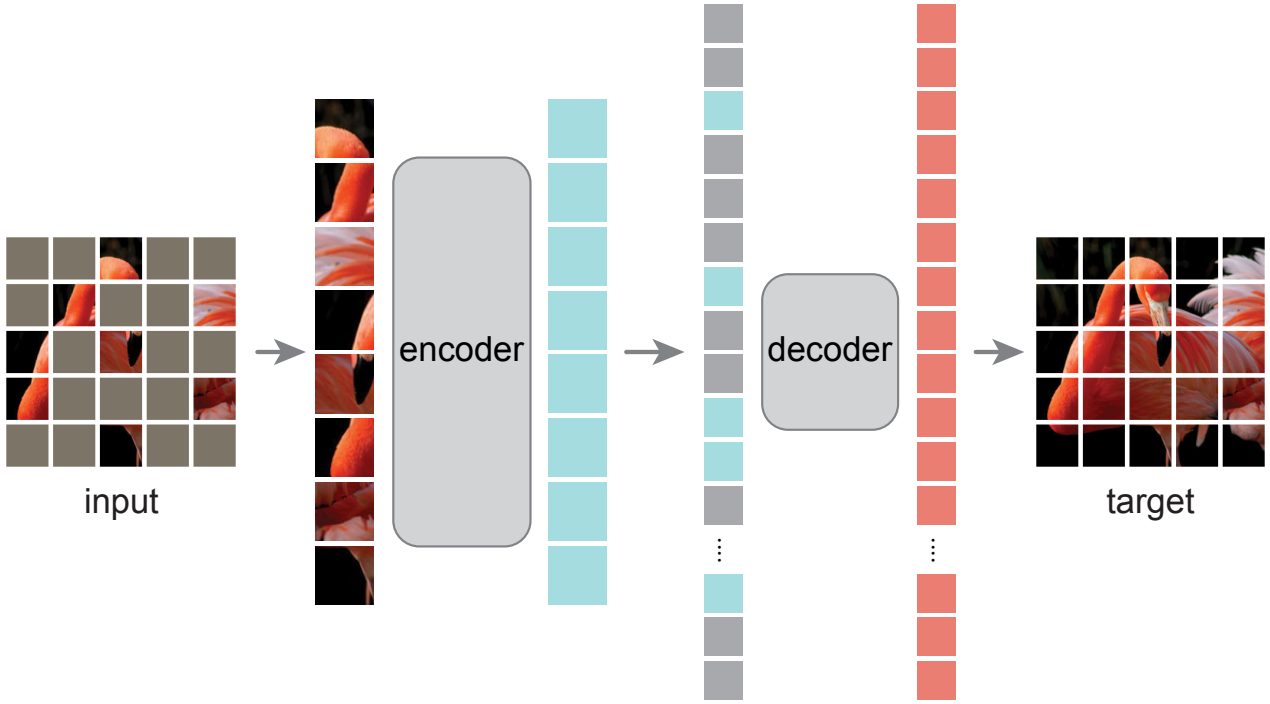



Image source: Kaiming He and Xinlei Chen and Saining Xie and Yanghao Li and Piotr Dollár and Ross Girshick, CC BY 4.0 (https://creativecommons.org/licenses/by/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Masked_Autoencoder.svg

# Masked Autoencoder: Core Idea

The Masked Autoencoder (MAE, He et al., 2022) is a **self-supervised pre-training** method that learns useful visual representations by solving a reconstruction task:

1. **Mask** a large fraction (typically 75%) of the $N$ image patches at random
2. **Encode** only the **visible** (unmasked) patches using the full ViT encoder — this is the key efficiency gain
3. **Decode** all $N$ patch positions (visible + masked) using a **lightweight decoder** ViT, predicting the pixel content of the masked patches

Since the encoder only processes $\sim 25\%$ of patches, MAE pre-training is computationally efficient despite the large ViT backbone.

# MAE: Asymmetric Encoder–Decoder

**Encoder (large, full ViT):**

- Receives only the visible patch tokens (no mask tokens)
- Produces rich representations of visible patches
- This is the part that is used after pre-training

**Decoder (small, lightweight ViT):**

- Receives encoded visible patches + learnable mask tokens at masked positions
- Reconstructs the pixel content of all $N$ patches
- Discarded after pre-training — only the encoder weights are kept

The asymmetry means the heavy encoder never processes uninformative mask tokens.

# MAE: Reconstruction Objective

MAE predicts the **normalized pixel values** of the masked patches.

Each patch is divided into $P \times P$ pixels. The decoder outputs a vector of size $P^2C$ per patch position, compared against the ground-truth patch pixels.

The training loss is the **mean squared error (MSE)** computed only over the masked patches:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \left\| \hat{\mathbf{p}}_i - \mathbf{p}_i \right\|^2$$

where $\mathcal{M}$ is the set of masked patch indices, $\mathbf{p}_i$ is the normalized ground-truth patch, and $\hat{\mathbf{p}}_i$ is the decoder prediction.

# Masked Autoencoder Examples

Examples from the ImageNet dataset:

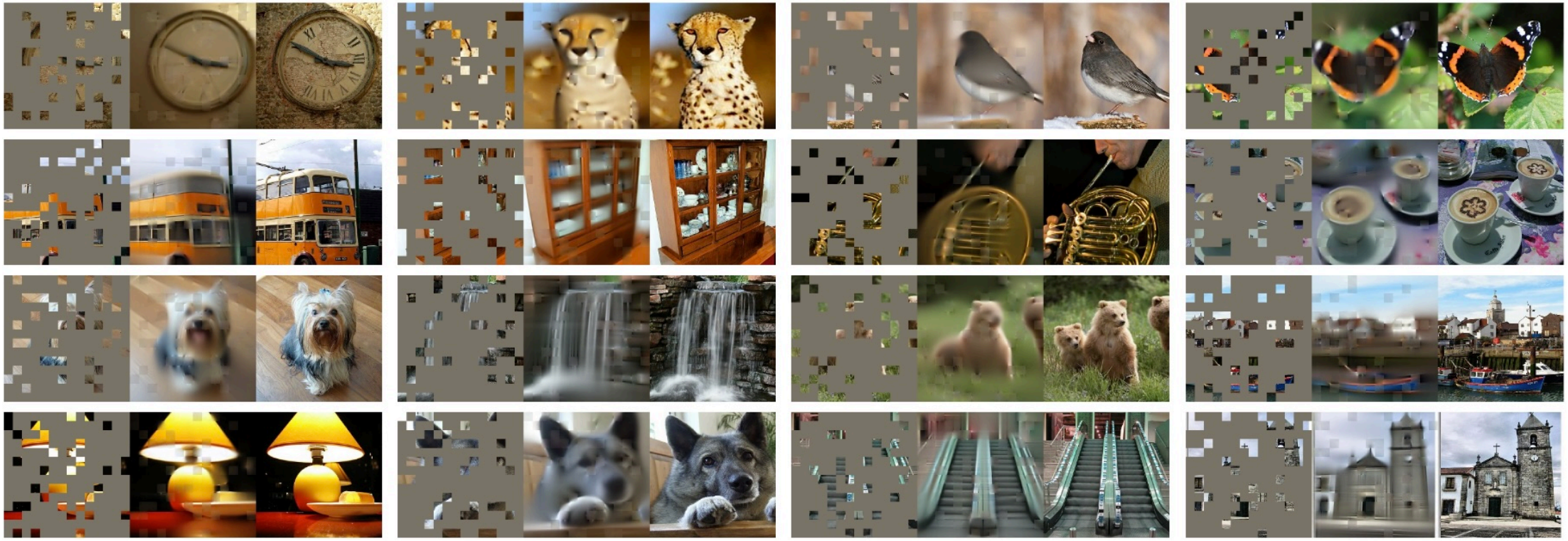


Image source: He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 16000-16009).

# Masked Autoencoder Examples

Examples from the COCO dataset:



Image source: He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 16000-16009).

# MAE: Why High Masking Ratio?

Masking 75% of patches is key:

- Low masking → reconstruction is too easy; patches can be reconstructed from adjacent visible patches by simple interpolation
- High masking → the model must **understand global image structure** to fill in large missing regions
- The task is genuinely hard only when context is sparse: the model must learn meaningful semantic representations, not just local texture matching

This high masking ratio also enables the encoder to run on only ~25% of the patches per image, dramatically reducing compute during pre-training.

# SimCLR Transformations

Transformations $\mathcal{T}$ used in the original SimCLR paper:

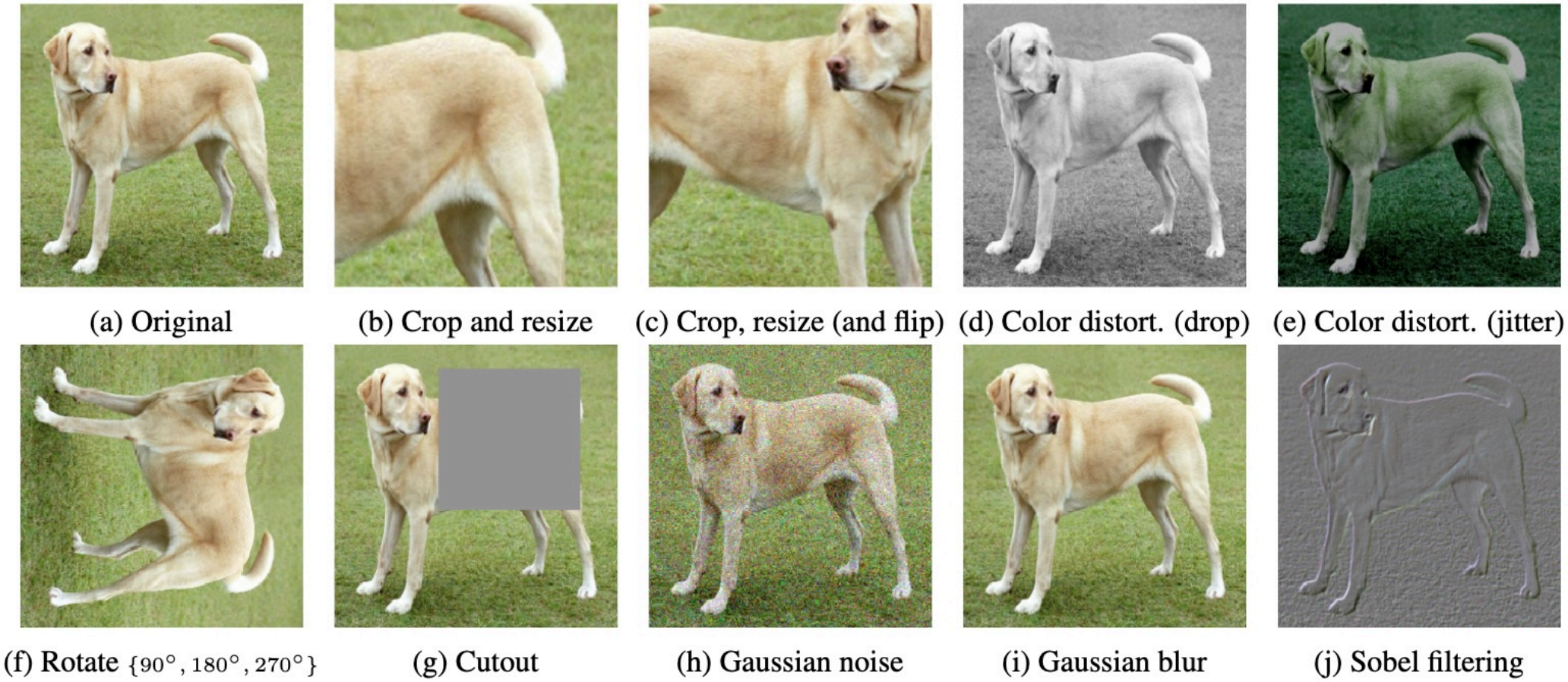



Image source: Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020, November). A simple framework for contrastive learning of visual representations. In International conference on machine learning (pp. 1597-1607). PmLR.

# SimCLR Pre-Training

Transformations $t$ and $t'$ are both sampled from the same type of transformations $\mathcal{T}$:

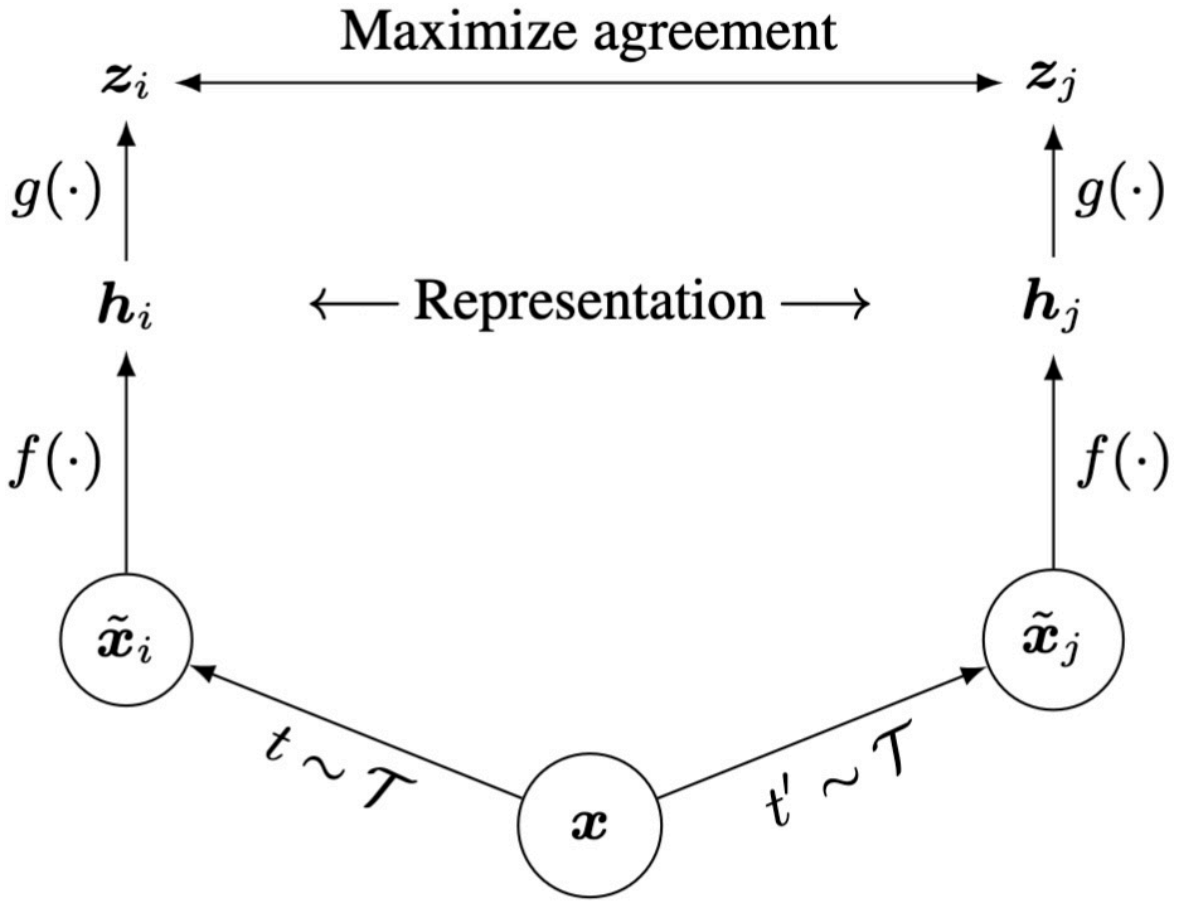



Image source: Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020, November). A simple framework for contrastive learning of visual representations. In International conference on machine learning (pp. 1597-1607). PmLR.

# SimCLR: Contrastive Objective

SimCLR (Chen et al., 2020) trains a visual encoder $f_\theta$ using a **contrastive loss** that pulls together different augmented views of the same image (positive pairs) and pushes apart views from different images (negative pairs).

For a mini-batch of $N$ images, each augmented twice, the **normalized temperature-scaled cross-entropy (NT-Xent) loss** for a positive pair $(i, j)$ is:

$$\mathcal{L}_{ij} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}$$

where $\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$ is cosine similarity, $\tau$ is a temperature hyperparameter, and $\mathbf{z} = g(f_\theta(\mathbf{x}))$ is the projection of the encoder output through a small MLP head $g$.

# SimCLR with a ViT Backbone

Originally proposed with a ResNet backbone, SimCLR can equally use a **ViT backbone**:

- The ViT encoder $f_\theta$ processes augmented patch sequences
- The class token output $\mathbf{z}_{\text{cls}}^{(L)}$ serves as the image representation
- A projection MLP head maps it to the contrastive embedding space

After pre-training, the projection head is discarded and the encoder is fine-tuned on a downstream task with labels.

# Contrastive Language-Image Pretraining

Pretraining using semantic similarities between images and text:

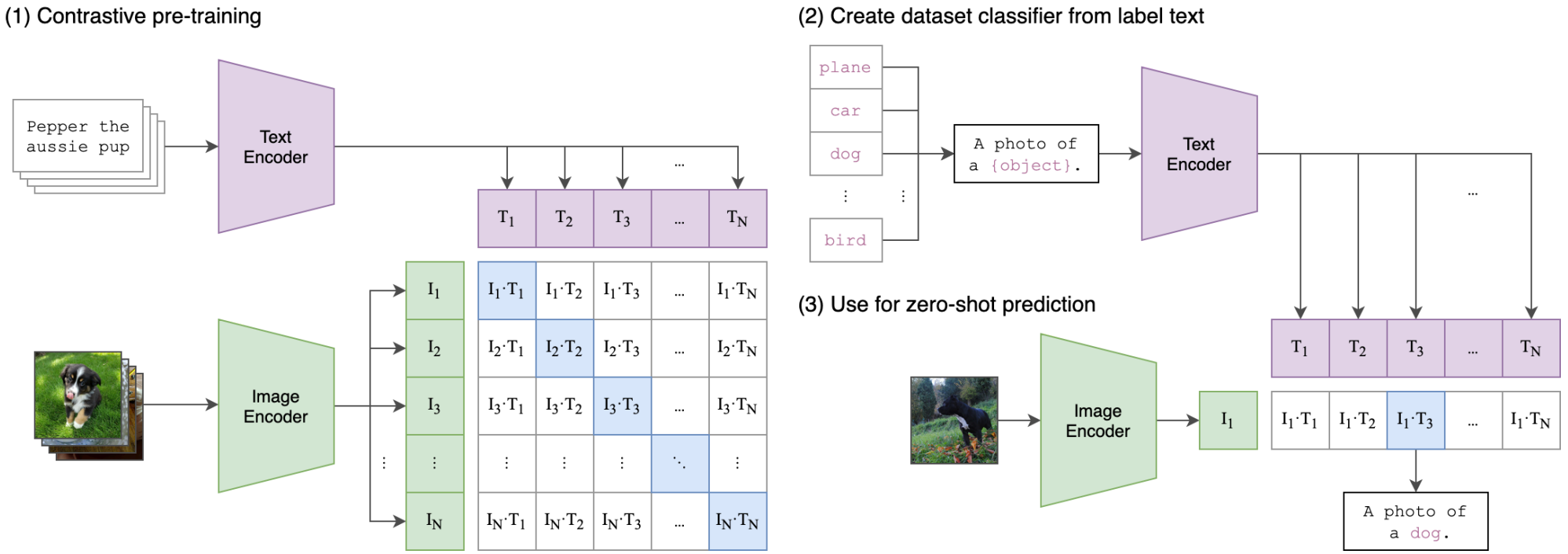



Image source: OpenAI, MIT , via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Contrastive_Language-Image_Pretraining.png

# CLIP: Architecture

**CLIP** (Contrastive Language–Image Pre-training, Radford et al., 2021) trains two separate encoders:

- **Image encoder** $f_\theta$: a ViT (or ResNet) that maps an image to an embedding $\mathbf{v} = f_\theta(\mathbf{x}) \in \mathbb{R}^d$
- **Text encoder** $g_\phi$: a Transformer that maps a text caption to an embedding $\mathbf{u} = g_\phi(t) \in \mathbb{R}^d$

Both embeddings are $L_2$-normalized before computing similarity. The model is trained on image–caption pairs collected from the internet.

# CLIP: Training Objective

For a mini-batch of $N$ image–caption pairs $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N}$:

$$\mathbf{v}_i = \frac{f_\theta(\mathbf{x}_i)}{\|f_\theta(\mathbf{x}_i)\|}, \quad \mathbf{u}_i = \frac{g_\phi(t_i)}{\|g_\phi(t_i)\|}$$

The similarity matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$ with $S_{ij} = \mathbf{v}_i^\top \mathbf{u}_j / \tau$ is computed, and the loss maximizes the diagonal entries (correct pairs):

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}[\mathcal{L}_{\text{img}\to\text{txt}} + \mathcal{L}_{\text{txt}\to\text{img}}]$$

where each term is a cross-entropy loss over the $N$ rows or columns of $\mathbf{S}$ with the identity permutation as the ground-truth assignment.

# CLIP: The Two Directional Losses

**Image → Text loss** $\mathcal{L}_{\text{img}\to\text{txt}}$: For each image $i$, find its matching caption among all $N$ texts in the batch:

$$\mathcal{L}_{\text{img}\to\text{txt}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ij})}$$

*"Given image $i$, which of the $N$ texts belongs to it?"* — softmax is computed across the columns of row $i$.

# CLIP: The Two Directional Losses

**Text → Image loss** $\mathcal{L}_{\text{txt}\to\text{img}}$: For each caption $i$, find its matching image among all $N$ images in the batch:

$$\mathcal{L}_{\text{txt}\to\text{img}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp(S_{ii})}{\sum_{j=1}^{N}\exp(S_{ji})}$$

*"Given text $i$, which of the $N$ images belongs to it?"* — softmax is computed across the rows of column $i$.

# CLIP: The Two Directional Losses

The final loss $\mathcal{L}_{\text{CLIP}} = \frac{1}{2}[\mathcal{L}_{\text{img}\to\text{txt}} + \mathcal{L}_{\text{txt}\to\text{img}}]$ is **symmetric**: both encoders are trained jointly, pushing matching pairs together and pushing all $N^2 - N$ non-matching pairs apart.

# CLIP: Zero-Shot Inference

A key capability of CLIP is **zero-shot image classification** — no task-specific fine-tuning is needed:

1. Define $K$ class names: `"a photo of a cat"`, `"a photo of a dog"`, …
2. Encode all $K$ text prompts with the text encoder: $\mathbf{u}_1, \ldots, \mathbf{u}_K$
3. Encode the query image: $\mathbf{v}$
4. Predict the class with the highest cosine similarity:

$$\hat{k} = \arg\max_k \mathbf{v}^\top \mathbf{u}_k$$

This works because both encoders are trained to align images and their natural-language descriptions in the same embedding space.

# ViT as a Dense Feature Extractor

Beyond classification, ViT is increasingly used as a **dense feature extractor** for spatial tasks:

**How it works:**

- After the last encoder layer, the $N$ patch token representations $\mathbf{z}_1^{(L)}, \ldots, \mathbf{z}_N^{(L)}$ are reshaped to a 2D feature map of size $\frac{H}{P} \times \frac{W}{P} \times d$
- This feature map is fed to a **task-specific head** — a decoder, segmentation head, or depth regression network

# DINOv2: Motivation

CLIP requires paired image–text data. Can we pre-train a strong visual encoder using **only images** — without any text supervision or labels?

**DINO** (Caron et al., 2021) introduced self-distillation with no labels for ViT. **DINOv2** (Oquab et al., 2023) scales this idea with:

- A **curated high-quality dataset** (LVD-142M: 142 million images)
- **Improved training recipe**: distillation + masked image modeling combined
- Resulting in **general-purpose visual features** that transfer to dense prediction tasks (depth estimation, segmentation) with frozen backbones and simple task heads or probes

# DINOv2: Self-Distillation Principle

DINOv2 uses a **student–teacher framework** without any labels:

- **Student network** $f_s$: updated by gradient descent
- **Teacher network** $f_t$: updated by an **exponential moving average (EMA)** of the student weights:

$$\theta_t \leftarrow \lambda\theta_t + (1 - \lambda)\theta_s$$

The teacher is never trained directly — it is a slowly changing copy of the student. This provides **stable training targets** and prevents representational collapse.

The student is trained to predict the teacher's output for different augmented views of the same image — a form of **self-distillation**.

# DINOv1: Method and Loss

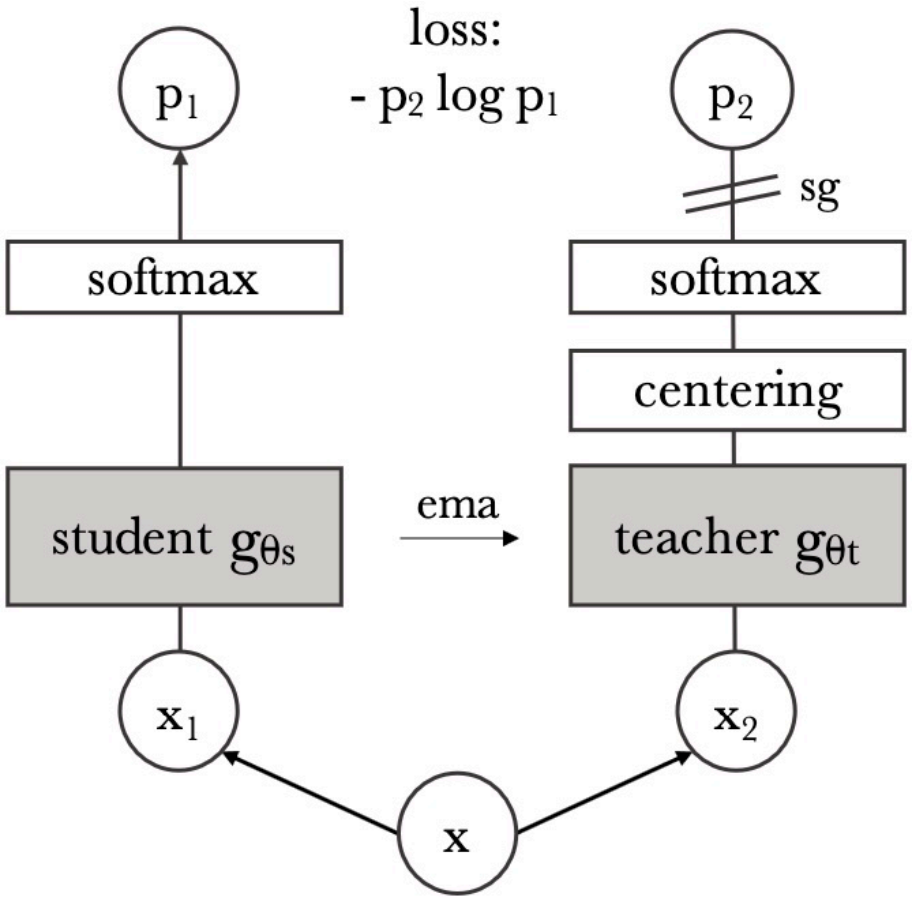



Image source: Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., & Joulin, A. (2021). Emerging properties in self-supervised vision transformers. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 9650-9660).

# DINOv2: Training Objective

DINOv2 trains the student to match the teacher's output distributions for different augmented views of the same image. The total loss combines three terms:

**DINO loss** (image-level, class token): cross-entropy between student and teacher softmax probability vectors $\mathbf{p}_s, \mathbf{p}_t$ over a shared codebook, summed over all crop pairs. Global crops go to both networks; local crops (small views, $\sim 20\%$) only to the student — so the student must predict global semantics from local context:

$$\mathcal{L}_{\text{DINO}} = -\sum_{\text{crop pairs}} \sum_{k} p_{t,k} \log p_{s,k}$$

# DINOv2: Training Objective

**iBOT loss** (patch-level): applies the same cross-entropy distillation to individual masked patch tokens, giving the model spatially structured representations.

**KoLeo regularization:** encourages diverse embeddings across mini-batch samples to prevent collapse.

**Overall loss in DINOv2:**

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{DINO}} + \lambda_{\text{iBOT}}\,\mathcal{L}_{\text{iBOT}} + \lambda_{\text{KoLeo}}\,\mathcal{L}_{\text{KoLeo}}$$

# DINOv2: Curated Dataset LVD-142M

A key contribution of DINOv2 is the **curated training dataset** LVD-142M:

- Collected from publicly available image datasets and the web
- **Deduplication**: near-duplicate images are removed to prevent memorization
- **Curation via retrieval**: images from small curated seed sets (e.g., ImageNet) are used to retrieve semantically similar images from a large uncurated pool — ensuring thematic diversity and quality
- Final dataset: **142 million high-quality images** covering a wide range of visual concepts

This curation process avoids the noise inherent in raw web-scraped data while remaining label-free — the model never sees any class annotations during pre-training.

# DINOv2: Curated Dataset LVD-142M

Data processing for collecting the LVD-142M dataset:

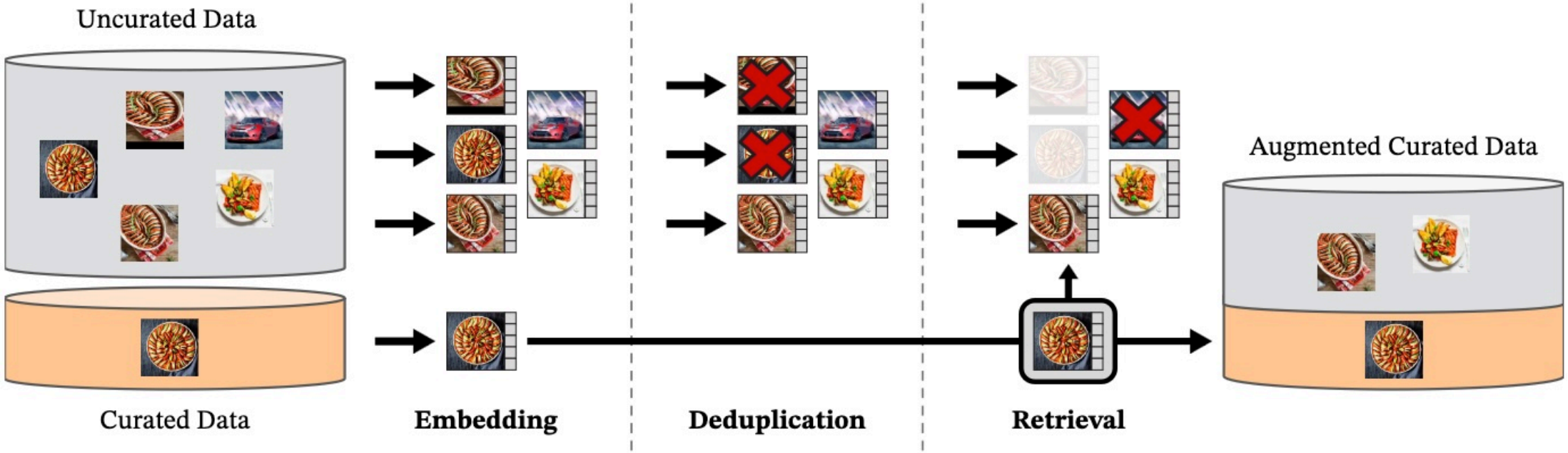



Image source: Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193.

# DINOv2: Feature Quality

DINOv2 produces **general-purpose visual features** that can be used as frozen backbone features:

- **Semantic segmentation**: patch token features can be clustered to segment objects without any segmentation supervision — the features are already semantically organized in space
- **Depth estimation**: simple heads on DINOv2 patch features achieve strong monocular depth estimation results
- **Classification**: a linear classifier on top of frozen DINOv2 features achieves strong results across many benchmarks

# DINOv2: Feature Quality

First components of PCA on the encoded feature patches:

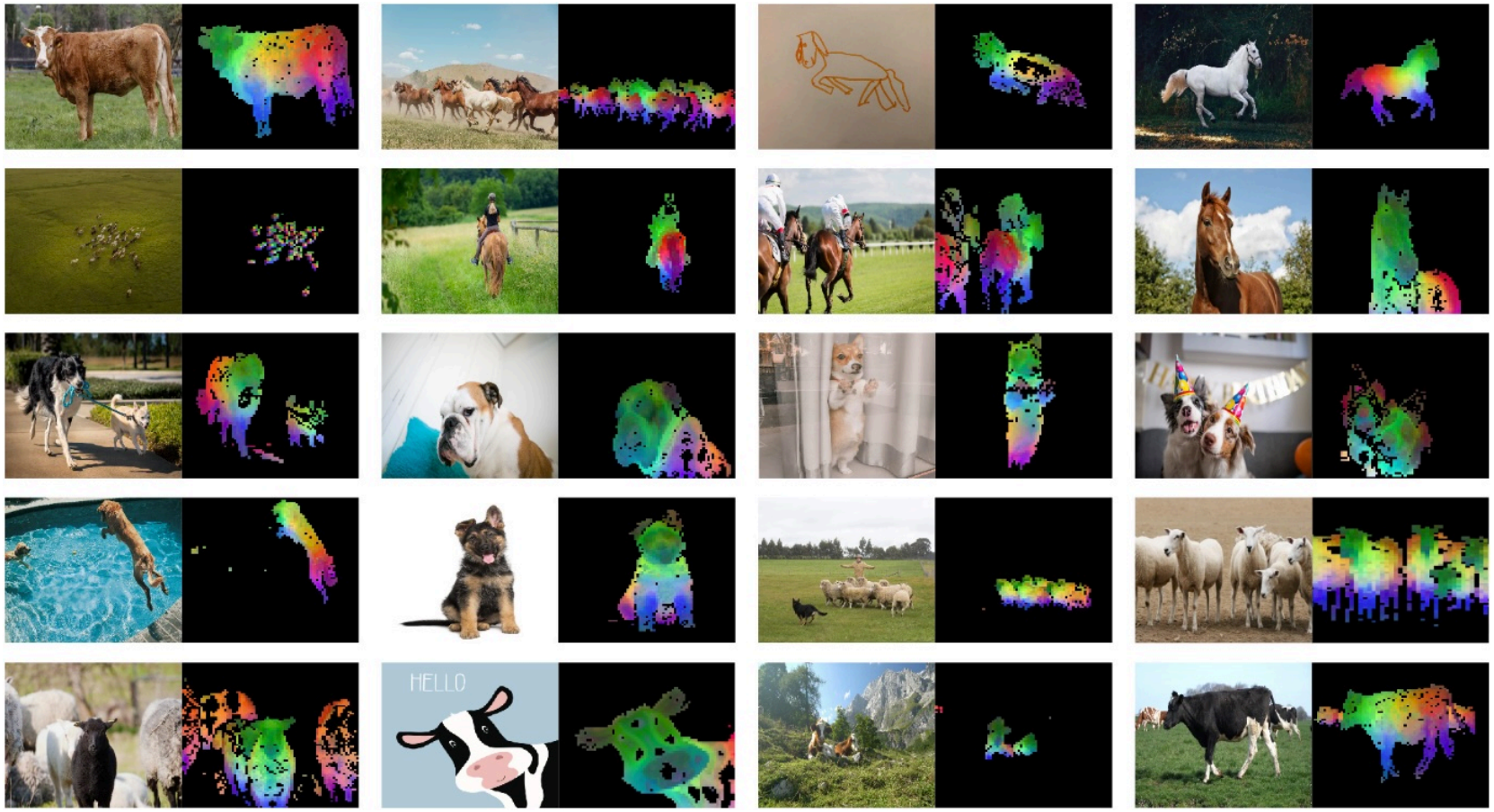



Image source: Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193.

# DINOv2: Dense Prediction With Frozen Backbones

A property of DINOv2 is that its patch token representations are **spatially structured**:

- Patches in semantically similar regions produce similar feature vectors
- The 2D layout of patch features resembles a semantic segmentation map — before any task-specific training

This allows dense prediction tasks (segmentation, depth) to be solved with a simple probe or task head applied to the patch token grid while keeping the backbone frozen.

This is in contrast to supervised classification pre-training where the patch tokens are not explicitly trained to be spatially meaningful — only the class token is trained.

# DINOv2: Dense Prediction With Frozen Backbones

Examples for semantic segmentation and depth estimation on the feature maps:

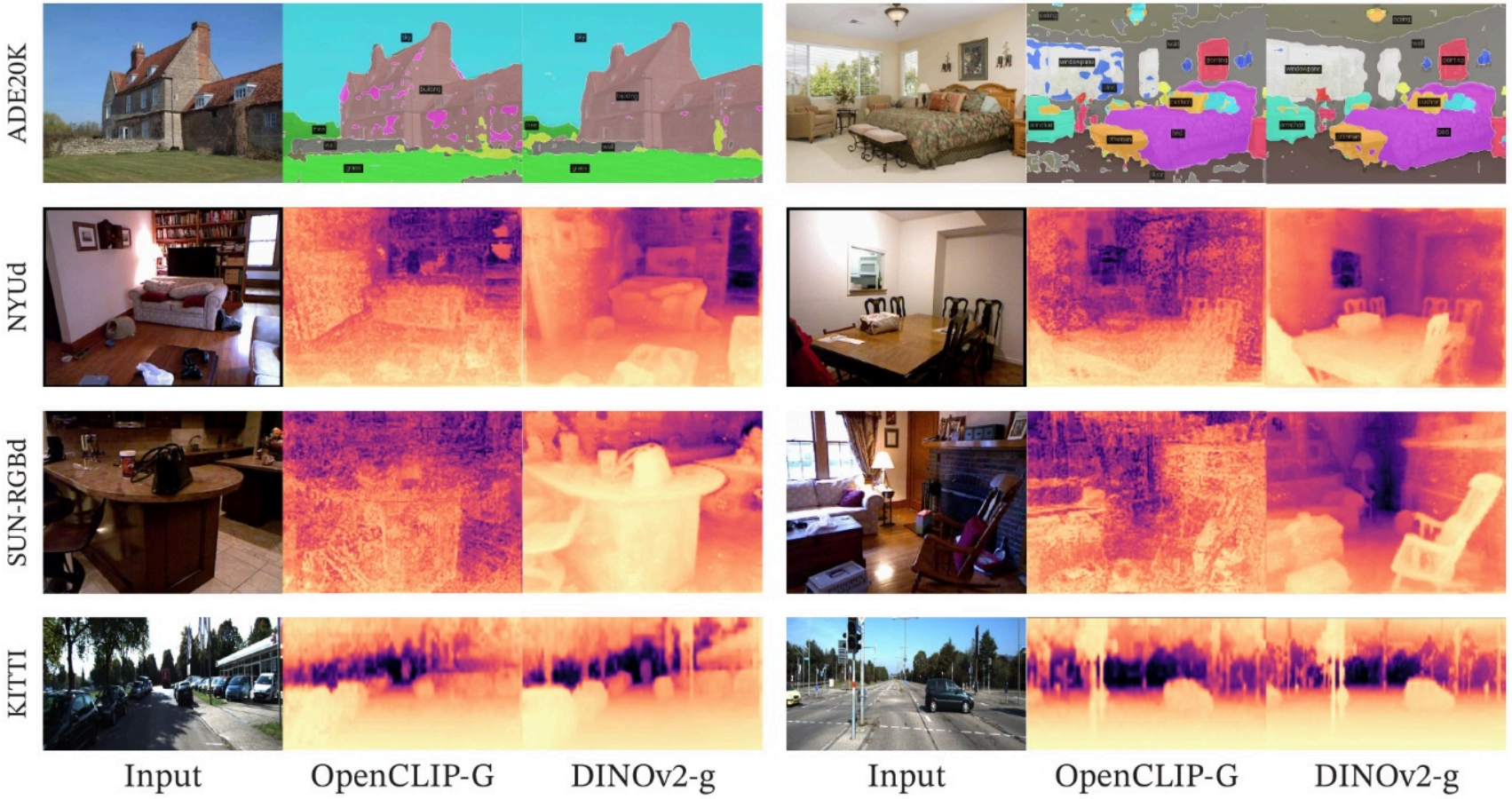



Image source: Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). Dinov2: Learning robust visual features without supervision. arXiv preprint arXiv:2304.07193.

# Applications of Pre-Trained ViT Features

A pre-trained ViT can be used for many downstream tasks:

| Task | How ViT features are used |
|---|---|
| Image classification | Linear probe or fine-tune on class token |
| Object detection | Patch token grid as backbone for region proposals |
| Semantic segmentation | Linear probe or decoder head on patch token grid |
| Monocular depth estimation | Linear probe or small decoder on patch tokens |
| Image retrieval | Cosine similarity in class token embedding space |
| Few-shot learning | Metric learning in frozen embedding space |

# Pre-Training Strategies: Summary

| Method | Supervisory Signal | Data Required | Key Strength |
|---|---|---|---|
| Supervised | Class labels | Labeled images | Strong classification features |
| MAE | Pixel reconstruction | Unlabeled images | Efficient, scalable |
| SimCLR | View consistency | Unlabeled images | No labels, strong representations |
| CLIP | Image–text alignment | Image–caption pairs | Zero-shot transfer, multimodal |
| DINOv2 | Self-distillation | Curated unlabeled images | Dense spatial features, no labels |

# Vision Transformer: Chapter Summary

**What we covered in this chapter:**

1. **From tokens to patches** — images are split into fixed-size patches, linearly projected to patch embeddings, and processed as a sequence
2. **ViT architecture** — patch embedding, class token, 1D learnable positional encoding, stacked encoder blocks, MLP classification head
3. **Self-attention on patches** — every patch attends to every other patch; multi-head attention captures diverse spatial relationships; attention maps are interpretable
4. **CNN vs. ViT** — ViT lacks spatial inductive bias but scales better with data
5. Pretraining strategies: **supervised**, **MAE**, **SimCLR**, **CLIP** and **DINOv2**